Building a Lithuanian "Contexto" Game

A Complete Blueprint from NLP Processing to Cloud Deployment

This technical blueprint outlines the architecture, data structures, implementation algorithms, and deployment configurations required to build a semantic proximity game equivalent to Contexto.me optimized specifically for the Lithuanian language.

1. Technical Challenges of the Lithuanian Language

Unlike English, Lithuanian is a highly inflected, synthetic Baltic language characterized by rich morphology and complex grammatical mutations. Implementing a semantic search or vector proximity game in Lithuanian poses distinct challenges:

- **Morphological Variations:** A single Lithuanian noun (e.g., `šuo`) can change its ending through 7 grammatical cases, both singular and plural (e.g., `šuns, šuniui, šunį, šunimi, šunyje, šunys, šunų, šunims, šunis, šunimis, šunyse, šunie`).
- **Prefixes and Suffixes:** Verbs modify their context dynamically based on aspects and tenses, creating a highly sparse distribution if raw tokens are analyzed.

> **Critical Architectural Rule:** If raw strings are fed directly into a vector space without normalization, guessing variations of the same word will yield erratic results. Input words must be systematically resolved to their primary singular nominative form for nouns, or infinite form for verbs, a process known as **Lemmatization**.

2. Processing Architecture & Core Engine

The processing architecture operates via a decoupled microservice model: a lightweight client frontend communicates asynchronously with a heavy, RAM-optimized Python backend holding the natural language model vectors.

The Vector Math Mechanism

Words are mapped mathematically as multidimensional spatial coordinates. If A represents the user's guessed word vector and B represents the hidden target word vector, proximity is determined by calculating the cosine of the angle between them:

$$\text{Cosine Similarity} = (A \cdot B) / (||A|| \times ||B||)$$

The resulting scalar value falls between -1.0 and 1.0. A similarity near 1.0 signifies maximum contextual proximity.

3. Phase-by-Phase Step Guide

1 Data Acquistion and Optimization

To power the geometric engine, we use pre-trained semantic vector sets from Meta's **fastText** initiative, which are superior for inflected languages due to sub-word n-gram training.

1. Download the Lithuanian binary model dataset (`cc.lt.300.bin`) from the official fastText open-source portal. Note that the uncompressed raw binary requires roughly 4.2 GB of disk storage and RAM space.
2. Filter out noise, typos, and foreign alpha-numeric characters from the model vocab to create a highly optimized subset consisting of the top 30,000–50,000 most frequently utilized Lithuanian vocabulary lemmas.

2 Developing the Python Backend Engine

The backend requires high memory availability to maintain the fastText matrix model within RAM. Create an app directory structure comprising a Python controller script (`main.py`), environment setup instructions (`requirements.txt`), and an OS virtualization package (`Dockerfile`).

Code Implementation: `main.py`

This script constructs a high-performance REST API using **FastAPI**, initializes Stanford's **Stanza NLP** pipeline for deep Lithuanian tokenization, and loads the vector model.

```
import os
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from gensim.models import FastText
import stanza

app = FastAPI(title="Lithuanian Contexto Game Engine API")

# Configure Cross-Origin Resource Sharing (CORS) for Frontend access
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Restrict to specific domains in production environment
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Initialize models globally on startup
print("[STATUS] Booting and downloading Stanza Lithuanian pipeline...")
stanza.download('lt', processors='tokenize,pos,lemma')
nlp = stanza.Pipeline('lt', processors='tokenize,pos,lemma')

print("[STATUS] Loading fastText Lithuanian Vector Binary (4.2GB)...")
# Ensure the cc.lt.300.bin file is placed in your workspace root
model = FastText.load_fasttext_format('cc.lt.300.bin')

# Statically defined target word for illustration (e.g., 'krepšinis' -> basketball)
TARGET_WORD_LEMMA = "krepšinis"

@app.get("/api/v1/guess")
async def process_user_guess(word: str):
    clean_word = word.strip().lower()
    if not clean_word:
        raise HTTPException(status_code=400, detail="Word parameter cannot be empty.")

    # Run the raw input string through the Stanza pipeline
    doc = nlp(clean_word)

    try:
        # Extract the resolved lemma from the tokenized sentence structure
        resolved_lemma = doc.sentences[0].words[0].lemma
    except Exception:
        raise HTTPException(status_code=422, detail="Unable to parse input text morphology.")

    # Compute the cosine distance against the target word
    try:
        similarity_score = float(model.wv.similarity(resolved_lemma, TARGET_WORD_LEMMA))

        # Format response packet
        return {
            "input": word,
            "lemma": resolved_lemma,
```

```
            "similarity": round(similarity_score, 5),
            "status": "success"
        }
    except KeyError:
        # Occurs if the lemma is completely missing from the fastText training corpus
        raise HTTPException(status_code=404, detail="Žodis nerastas duomenų bazėje.")
```

Environment Definition: `requirements.txt`

List the exact pinning configurations for the native dependencies to ensure reproducible cloud builds.

```
fastapi>=0.100.0
uvicorn>=0.22.0
gensim>=4.3.1
stanza>=1.5.0
scipy>=1.10.1
```

Infrastructure Virtualization: `Dockerfile`

Because hosting large files introduces platform-specific bugs, a standardized Docker image provides full control over structural OS conditions.

```
FROM python:3.9-slim

WORKDIR /app

# Install system dependencies needed for compiling certain python binary extensions
RUN apt-get update && apt-get install -y --no-install-recommends     build-essential     && rm -
rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip &&     pip install --no-cache-dir -r
requirements.txt

# Pre-stage Stanza language files into the image directory during step generation
RUN python -c "import stanza; stanza.download('lt', processors='tokenize,pos,lemma')"

COPY . .

EXPOSE 7860

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

3 Backend Infrastructure & Hosting Selection

Standard container options like Render or Fly.io free tiers cap available memory at 512MB, which will instantly trigger an *Out-Of-Memory (OOM) Kill* error when loading the fastText dataset.

The optimal deployment host is **Hugging Face Spaces** running a **Docker SDK instance**. Their persistent free CPU tier supplies **2 vCPUs along with 16 GB of RAM**, perfectly accommodating the memory fingerprint of our platform.

Hosting Infrastructure	RAM Allocation	Storage Volume Limit	Applicability for Project
Render / Railway (Free)	512 MB	Shared / Ephemeral	Incompatible (Instant OOM Crash)
AWS EC2 t3.medium	4.0 GB	EBS Configurable	Compatible (Paid Tier)
Hugging Face Spaces (Free CPU)	**16.0 GB**	**50 GB Container Space**	**Highly Recommended (Free & Reliable)**

Deployment Steps for Hugging Face Spaces:

1. Create a free account on Hugging Face and navigate to the "Spaces" dashboard.
2. Click "Create New Space", assign a unique workspace title, and select **Docker** as the environment platform SDK (choose the "Blank" template).
3. Commit and push your `main.py`, `requirements.txt`, `Dockerfile`, along with the large `cc.lt.300.bin` data file to the repository provided by Hugging Face via Git LFS.
4. The system will detect the structural `Dockerfile` configurations, spin up the dependencies, execute pre-downloads, and activate your API endpoint live under an public URL sequence (e.g., `https://user-space.hf.space/api/v1/guess`).

4 Developing and Hosting the Frontend Web Client

The user interface layout requires no active server processing power. It is composed entirely of client-side logic (HTML/CSS/JavaScript) hosted on platforms like **Vercel** or **Netlify**. Below is a native JavaScript code interface demonstrating connection parsing.

```
// Frontend Configuration Variables
const BACKEND_API_ENDPOINT = "https://your-custom-space.hf.space/api/v1/guess";

async function dispatchUserGuess(rawInputWord) {
    const sanitizedInput = rawInputWord.trim();
    if (!sanitizedInput) return;

    try {
        // Issue secure HTTP asynchronous call to the Hugging Face microservice
        const response = await fetch(`${BACKEND_API_ENDPOINT}?word=$
{encodeURIComponent(sanitizedInput)}`);

        if (!response.ok) {
            const problemReport = await response.json();
            throw new Error(problemReport.detail || "Server communication malfunction.");
        }

        const payload = await response.json();

        // Integration mapping with the Game UI state matrix
        appendGuessToGameBoard(payload.lemma, payload.similarity);

    } catch (networkError) {
        console.error("Game Loop Processing Failure:", networkError.message);
        displayUIErrorNotification(networkError.message);
    }
}

function appendGuessToGameBoard(lemma, similarityScore) {
    console.log(`Successfully parsed Lemma: ${lemma} with proximity metrics: ${similarityScore}
`);
    // Code instructions here to update progress bars, text elements, and color configurations
}
```

> **Production Enhancement Optimization Note:** Calculating semantic similarities using large vector models dynamically on every API request can introduce performance lag if user concurrency scales. To maximize rendering speeds, pre-compute the proximity ranking score matrix for all 40,000 dictionary lemmas against chosen target words beforehand. Store these pre-calculated index values inside an optimized key-value database schema (like SQLite or Redis) to enable immediate ***O(1)*** lookups.